# Technical Report: Design and Implementation of an ANN-Based Inverse Kinematics Model for a Robotic Arm

Course: AI Control

Institution: Banha University – Shoubra faculty of Engineering

Date: December 24, 2025

**Project Team:**

1. Soliman Mohamed Naguib
2. Aly Ahmed Aly Agamy
3. Mohamed Maher Abd ElMohsen Ali
4. Ahmed Mohamed Ibrahim
5. AbdelWahab Amr
6. Abdelrahman Mohamed AbdelMawgod
7. Ahmed Magdy
8. Karim Walid

Supervisor: Prof.Dr.Mahmoud ElSamanty

## 1. Introduction

Inverse kinematics (IK) is a fundamental problem in robotic manipulators, where the objective is to determine the joint angles required to achieve a desired end-effector position and orientation. Analytical solutions for inverse kinematics become increasingly complex or even infeasible for multi-degree-of-freedom robotic arms due to nonlinearity and redundancy.

In this work, an **Artificial Neural Network (ANN)** is employed as a data-driven approach to approximate the inverse kinematics mapping of a five-degree-of-freedom robot arm. The ANN learns the nonlinear relationship between the end-effector pose and the corresponding joint angles using supervised learning.

---

## 2. Dataset Description

The dataset is stored in an Excel file and contains multiple samples representing valid robot configurations within the reachable workspace.

Each sample consists of:

- **Inputs**: End-effector position and orientation parameters.
- **Outputs**: Joint angles $Q_1, Q_2, Q_3, Q_4, Q_5$.

The joint angles were initially stored in radians and later converted to degrees to improve interpretability and compatibility with practical robotic control systems.

---

## 3. Data Preprocessing

### 3.1 Joint Angle Conversion

All joint angles were converted from radians to degrees using:

$$\theta_{deg} = \theta_{rad} \times \frac{180}{\pi}$$

This conversion simplifies result interpretation and aligns with common robotic actuator interfaces.

---

### 3.2 Trigonometric Representation of Orientation

To improve learning performance, angular inputs were represented using their sine and cosine components:

$$\sin(\theta), \cos(\theta)$$

This representation avoids discontinuities caused by angle wrapping at $0°$and $360°$, allowing the neural network to learn smooth periodic relationships.

---

### 3.3 Feature Scaling

Both input and output data were normalized using **Standard Scaling**:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

Normalization ensures that all features contribute equally to the training process and prevents dominance of features with large numerical ranges.

Separate scalers were used for:

- Input features
- Output joint angles

These scalers were saved for consistent preprocessing during model inference.

---

## 4. Dataset Splitting

The dataset was divided into training and testing sets as follows:

- **80% Training Data**
- **20% Testing Data**

This split ensures reliable evaluation of the model's generalization capability.

---

## 5. Neural Network Architecture

The ANN was implemented using the **TensorFlow/Keras Sequential API**.
The architecture is summarized below:

| **Layer** | **Description** |
|---|---|
| Input Layer | Receives normalized end-effector features |
| Dense (256) | Fully connected layer with ReLU activation |
| Batch Normalization | Stabilizes and accelerates training |
| Dropout (0.2) | Reduces overfitting |
| Dense (256) | Nonlinear feature extraction |
| Batch Normalization | Improves convergence |
| Dropout (0.2) | Regularization |
| Dense (128) | High-level feature learning |
| Dense (5) | Output layer for joint angles |

The output layer uses a **linear activation function**, which is appropriate for regression problems.

---

### 6. Model Compilation

The model was compiled using the following configuration:

- **Optimizer**: Adam
- **Learning Rate**: 0.001
- **Loss Function**: Mean Squared Error (MSE)
- **Evaluation Metric**: Mean Absolute Error (MAE)

MSE penalizes large errors more strongly, while MAE provides an intuitive measure of average prediction error in joint angle units.

---

### 7. Training Strategy

The model was trained for a maximum of 500 epochs using a batch size of 64. Two callbacks were employed to improve training efficiency:

### 7.1 Early Stopping

Training is terminated when the validation loss stops improving, preventing overfitting and unnecessary computation.

### 7.2 Learning Rate Reduction

When validation performance plateaus, the learning rate is reduced to allow finer weight updates and improved convergence.

---

## 8. Model Saving and Deployment

After training, the following components were saved:

- **Trained ANN model** (.keras format)
- **Input scaler**
- **Output scaler**

Saving the scalers ensures that any new input data is preprocessed identically to the training data, which is critical for accurate predictions.

The trained model can be integrated into:

- ROS2 nodes
- Python-based control systems
- Robotic trajectory planning frameworks

---

## 9.code

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam
```

```python
drive.mount('/content/drive')
file_path = '/content/drive/MyDrive/Colab Notebooks/RobotArm_ANN/FixedReach_Workspace_Data.xlsx'
df = pd.read_excel(file_path)

output_cols = ['Q1', 'Q2', 'Q3', 'Q4', 'Q5']
df[output_cols] = np.degrees(df[output_cols])

df['R_dist'] = np.sqrt(df['X']**2 + df['Y']**2 + df['Z']**2)
df['r_xy'] = np.sqrt(df['X']**2 + df['Y']**2)

angle_features = ['Roll', 'Pitch', 'Yaw']
for col in angle_features:
    df[f'sin_{col}'] = np.sin(np.radians(df[col]))
    df[f'cos_{col}'] = np.cos(np.radians(df[col]))

feature_cols = ['X', 'Y', 'Z', 'R_dist', 'r_xy',
                'sin_Roll', 'cos_Roll',
```

```python
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(df[output_cols])

main_cluster_id = df['Cluster'].value_counts().idxmax()
df_train = df[df['Cluster'] == main_cluster_id].copy()

X = df_train[feature_cols].values
y = df_train[output_cols].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)

scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
y_train_scaled = scaler_y.fit_transform(y_train)
X_test_scaled = scaler_X.transform(X_test)
y_test_scaled = scaler_y.transform(y_test)
```

```
Mounted at /content/drive
```

```python
def build_model(input_dim, output_dim):
    model = Sequential([
        Input(shape=(input_dim,)),
        Dense(1024, activation='relu'),
        BatchNormalization(),
        Dropout(0.05),

        Dense(1024, activation='relu'),
        BatchNormalization(),
```

ANN Model.ipynb

```python
            Dense(512, activation='relu'),
            BatchNormalization(),

            Dense(256, activation='relu'),

            Dense(output_dim, activation='linear')
        ])
        return model

model = build_model(X_train_scaled.shape[1], y_train_scaled.shape[1])

optimizer = Adam(learning_rate=0.0005)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4, min_lr=1e-7)
early_stop = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)

model.compile(loss='mean_squared_error', optimizer=optimizer, metrics=['mae'])

history = model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_test_scaled, y_test_scaled),
    epochs=150,
    batch_size=32,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

```
Epoch 1/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 44s 28ms/step - loss: 0.3206 - mae: 0.4135 - val_loss: 0.1024 - val_mae: 0.2343 - learning_rate: 5.0000e-04
Epoch 2/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 71s 48ms/step - loss: 0.1092 - mae: 0.2442 - val_loss: 0.0662 - val_mae: 0.1839 - learning_rate: 5.0000e-04
Epoch 3/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 27ms/step - loss: 0.0840 - mae: 0.2101 - val_loss: 0.0572 - val_mae: 0.1655 - learning_rate: 5.0000e-04
Epoch 4/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 39s 26ms/step - loss: 0.0728 - mae: 0.1919 - val_loss: 0.0496 - val_mae: 0.1473 - learning_rate: 5.0000e-04
Epoch 5/150
```

ANN Model.ipynb

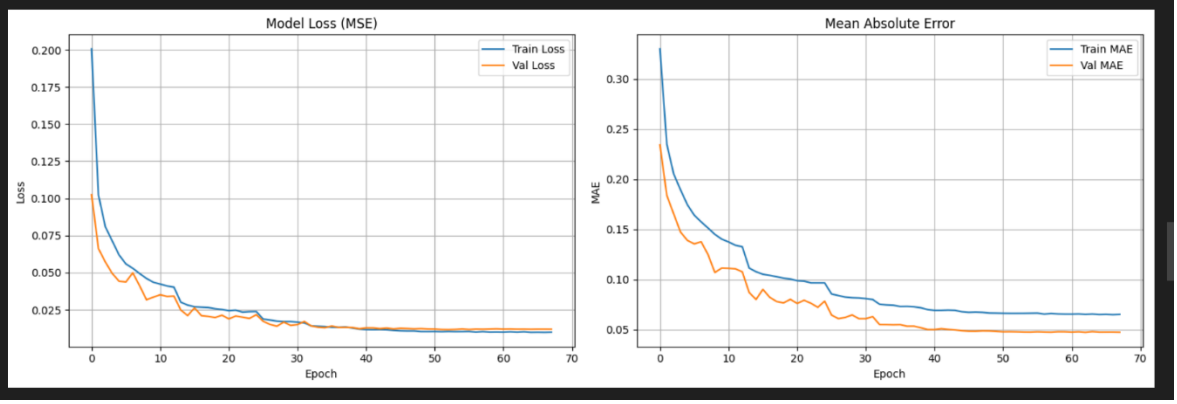
```
Epoch 1/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 44s 28ms/step - loss: 0.3206 - mae: 0.4135 - val_loss: 0.1024 - val_mae: 0.2343 - learning_rate: 5.0000e-04
Epoch 2/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 71s 48ms/step - loss: 0.1092 - mae: 0.2442 - val_loss: 0.0662 - val_mae: 0.1839 - learning_rate: 5.0000e-04
Epoch 3/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 51s 27ms/step - loss: 0.0840 - mae: 0.2101 - val_loss: 0.0572 - val_mae: 0.1655 - learning_rate: 5.0000e-04
Epoch 4/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 39s 26ms/step - loss: 0.0728 - mae: 0.1919 - val_loss: 0.0496 - val_mae: 0.1473 - learning_rate: 5.0000e-04
Epoch 5/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 38s 25ms/step - loss: 0.0620 - mae: 0.1755 - val_loss: 0.0443 - val_mae: 0.1390 - learning_rate: 5.0000e-04
Epoch 6/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 39s 26ms/step - loss: 0.0559 - mae: 0.1648 - val_loss: 0.0438 - val_mae: 0.1356 - learning_rate: 5.0000e-04
Epoch 7/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 42s 28ms/step - loss: 0.0530 - mae: 0.1581 - val_loss: 0.0499 - val_mae: 0.1377 - learning_rate: 5.0000e-04
Epoch 8/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 40s 27ms/step - loss: 0.0493 - mae: 0.1522 - val_loss: 0.0412 - val_mae: 0.1250 - learning_rate: 5.0000e-04
Epoch 9/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 38s 25ms/step - loss: 0.0460 - mae: 0.1449 - val_loss: 0.0318 - val_mae: 0.1071 - learning_rate: 5.0000e-04
Epoch 10/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 44s 27ms/step - loss: 0.0440 - mae: 0.1415 - val_loss: 0.0335 - val_mae: 0.1115 - learning_rate: 5.0000e-04
Epoch 11/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 37s 25ms/step - loss: 0.0431 - mae: 0.1385 - val_loss: 0.0351 - val_mae: 0.1112 - learning_rate: 5.0000e-04
Epoch 12/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 39s 26ms/step - loss: 0.0420 - mae: 0.1358 - val_loss: 0.0340 - val_mae: 0.1107 - learning_rate: 5.0000e-04
Epoch 13/150
...
Epoch 67/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 42s 28ms/step - loss: 0.0098 - mae: 0.0652 - val_loss: 0.0120 - val_mae: 0.0476 - learning_rate: 9.7656e-07
Epoch 68/150
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 40s 27ms/step - loss: 0.0099 - mae: 0.0657 - val_loss: 0.0119 - val_mae: 0.0475 - learning_rate: 9.7656e-07
```

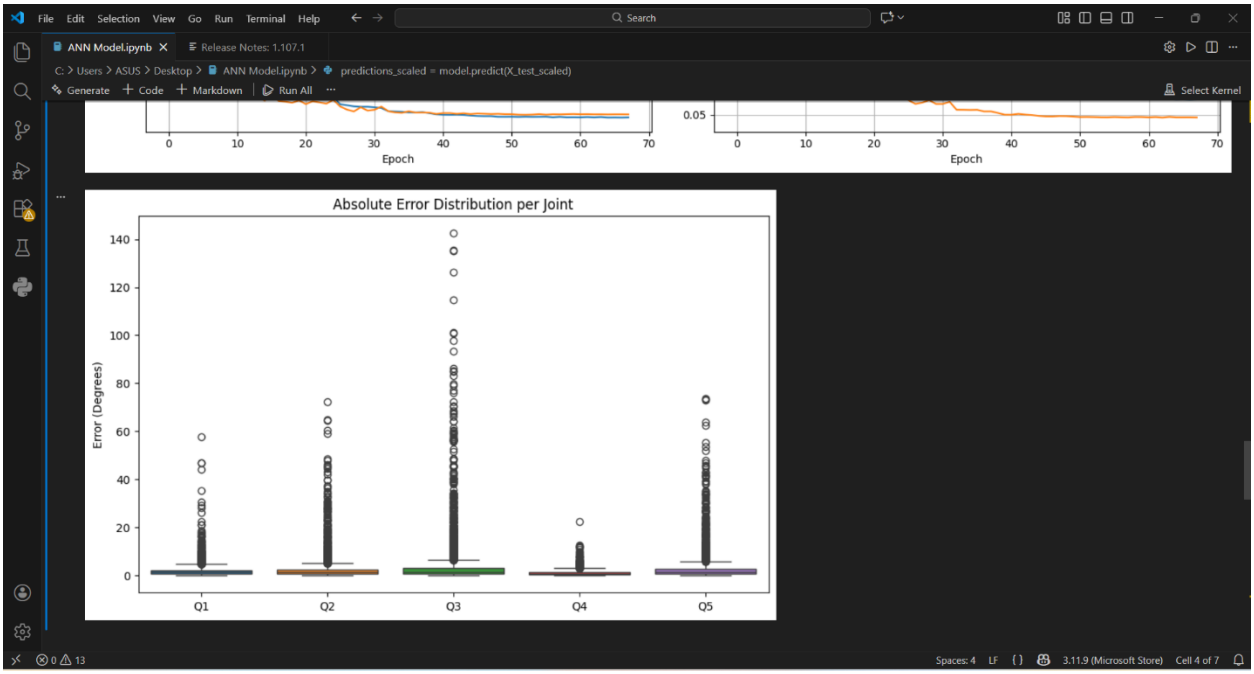
*Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...*

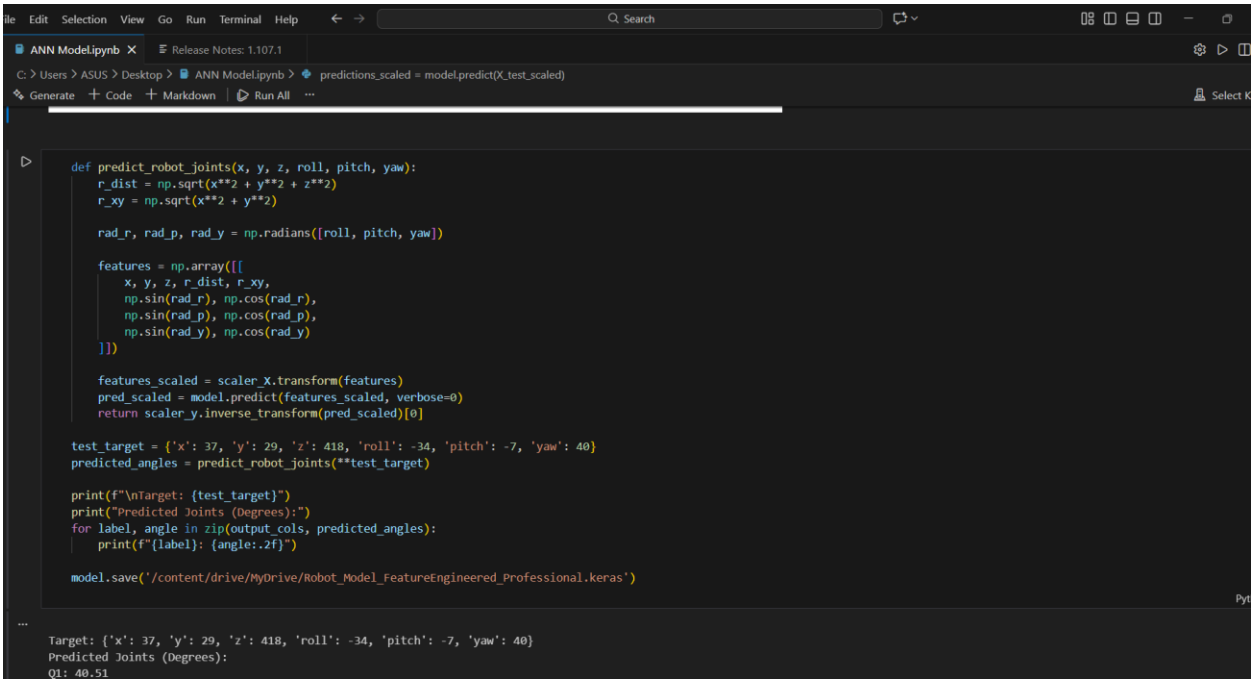
```python
predictions_scaled = model.predict(X_test_scaled)
predictions_degrees = scaler_y.inverse_transform(predictions_scaled)
y_test_degrees = scaler_y.inverse_transform(y_test_scaled)

abs_errors = np.abs(predictions_degrees - y_test_degrees)
mae_per_joint = np.mean(abs_errors, axis=0)
acc_per_joint = np.mean(abs_errors < 1.0, axis=0) * 100

print(f"{'Joint':<10} | {'MAE (deg)':<15} | {'Accuracy (<1°)'}")
print("-" * 45)
for i, joint in enumerate(output_cols):
    print(f"{joint:<10} | {mae_per_joint[i]:.4f}          | {acc_per_joint[i]:.2f}%")
print("-" * 45)
print(f"Overall Accuracy: {np.mean(abs_errors < 1.0) * 100:.2f}%")

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_title('Model Loss (MSE)')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history['mae'], label='Train MAE')
axes[1].plot(history.history['val_mae'], label='Val MAE')
axes[1].set_title('Mean Absolute Error')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('MAE')
axes[1].legend()
axes[1].grid(True)
```

```python
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
sns.boxplot(data=pd.DataFrame(abs_errors, columns=output_cols))
plt.title('Absolute Error Distribution per Joint')
plt.ylabel('Error (Degrees)')
plt.show()
```

```
167/167 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step
Joint      | MAE (deg)      | Accuracy (<1°)
---------------------------------------------
Q1         | 1.7750         | 38.08%
Q2         | 2.2837         | 36.80%
Q3         | 3.5113         | 32.62%
Q4         | 1.0592         | 58.62%
Q5         | 2.6609         | 33.45%
---------------------------------------------
Overall Accuracy: 39.91%
```




File Edit Selection View Go Run Terminal Help
Search
ANN Model.ipynb
Release Notes: 1.107.1
C: > Users > ASUS > Desktop > ANN Model.ipynb > predictions_scaled = model.predict(X_test_scaled)
Generate
Code
Markdown
Run All
Select Kernel
0.05
Epoch
Epoch
Absolute Error Distribution per Joint
Error (Degrees)
Q1
Q2
Q3
Q4
Q5
Spaces: 4
LF
3.11.9 (Microsoft Store)
Cell 4 of 7


ANN Model.ipynb
Release Notes: 1.107.1
C: > Users > ASUS > Desktop > ANN Model.ipynb > predictions_scaled = model.predict(X_test_scaled)
Generate
Code
Markdown
Run All
def predict_robot_joints(x, y, z, roll, pitch, yaw):
r_dist = np.sqrt(x**2 + y**2 + z**2)
r_xy = np.sqrt(x**2 + y**2)
rad_r, rad_p, rad_y = np.radians([roll, pitch, yaw])
features = np.array([[
x, y, z, r_dist, r_xy,
np.sin(rad_r), np.cos(rad_r),
np.sin(rad_p), np.cos(rad_p),
np.sin(rad_y), np.cos(rad_y)
]])
features_scaled = scaler_X.transform(features)
pred_scaled = model.predict(features_scaled, verbose=0)
return scaler_y.inverse_transform(pred_scaled)[0]
test_target = {'x': 37, 'y': 29, 'z': 418, 'roll': -34, 'pitch': -7, 'yaw': 40}
predicted_angles = predict_robot_joints(**test_target)
print(f"\nTarget: {test_target}")
print("Predicted Joints (Degrees):")
for label, angle in zip(output_cols, predicted_angles):
print(f"{label}: {angle:.2f}")
model.save('/content/drive/MyDrive/Robot_Model_FeatureEngineered_Professional.keras')
Target: {'x': 37, 'y': 29, 'z': 418, 'roll': -34, 'pitch': -7, 'yaw': 40}
Predicted Joints (Degrees):
Q1: 40.51

File Edit Selection View Go Run Terminal Help

ANN Model.ipynb | Release Notes: 1.107.1

C: > Users > ASUS > Desktop > ANN Model.ipynb > predictions_scaled = model.predict(X_test_scaled)

Generate + Code + Markdown | Run All ··· Select Kernel

```python
model.save('/content/drive/MyDrive/Robot_Model_FeatureEngineered_Professional.keras')
```

Python

```
Target: {'x': 37, 'y': 29, 'z': 418, 'roll': -34, 'pitch': -7, 'yaw': 40}
Predicted Joints (Degrees):
Q1: 40.51
Q2: -13.46
Q3: 11.03
Q4: -55.28
Q5: 83.92
```

Python

```python
save_path = '/content/drive/MyDrive/High_Precision_Robot_Model.keras'
model.save(save_path)

import joblib

joblib.dump(scaler_X, '/content/drive/MyDrive/scaler_X.pkl')

joblib.dump(scaler_y, '/content/drive/MyDrive/scaler_y.pkl')
```

Python

```
['/content/drive/MyDrive/scaler_y.pkl']
```

0 13 Spaces: 4 LF 3.11.9 (Microsoft Store) Cell 4 of 7

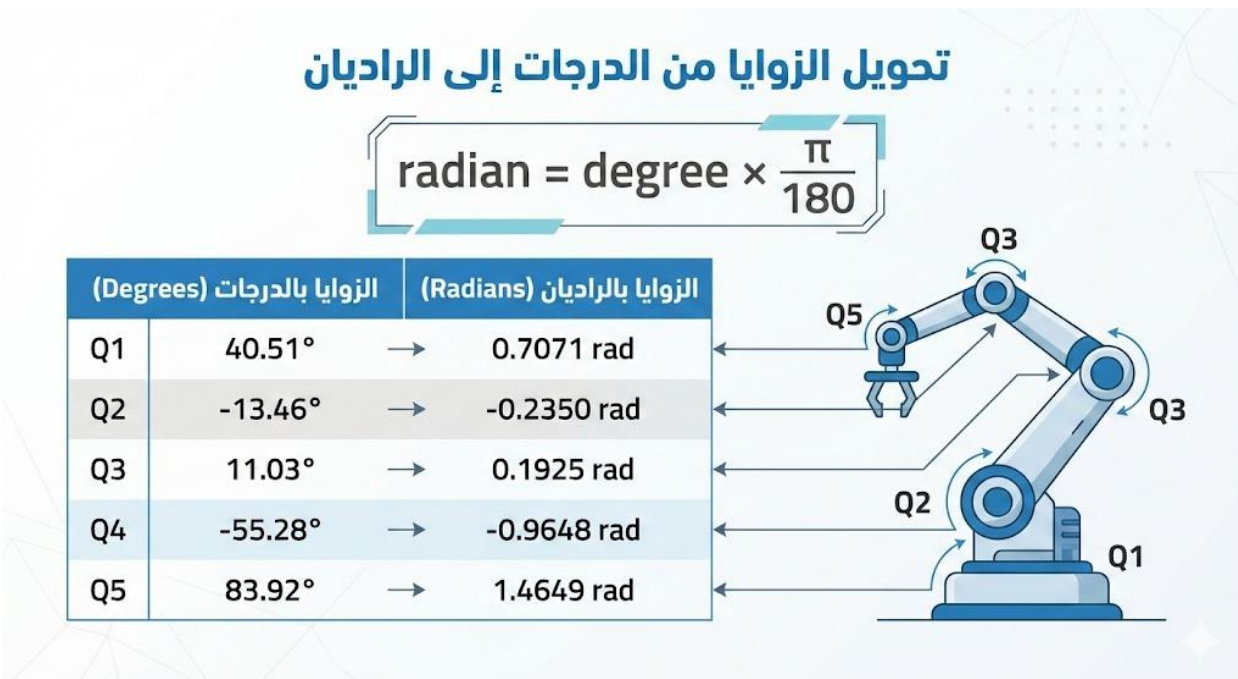
## 10.Results

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 16 | -118.3576437 | 170.7914346 | 149.2188346 | 65.32839175 | 16.80340675 | 68.051631 | -0.335722461 | -0.612704252 | -1.370482321 | 1.493193742 | -0.286934 |
| 17 | -56.68517724 | -171.2947437 | 37.74914589 | 169.9301092 | 74.62439022 | -7.35632754 | 1.074956123 | -0.832394505 | -1.216151031 | -0.603823373 | -1.11980 |
| 18 | 129.1501472 | -174.8392471 | 366.498645 | -107.4252512 | -18.85807541 | -50.5738018 | -0.999512296 | 1.240009497 | -1.186830924 | 0.287742269 | 1.171569 |
| 19 | 37.2114827 | 29.36933703 | 418.4738305 | -34.57499212 | -7.089344411 | 40.70240536 | 0.735557259 | -0.264219888 | 0.157772051 | -0.964458618 | 1.540403 |
| 20 | 219.3349942 | -72.4021458 | 280.8622691 | -152.8395784 | 45.57778408 | -10.01424683 | -0.408808888 | 0.582600524 | 0.277818351 | 1.26958052 | 1.40000 |
| 21 | -149.8101006 | -290.6574779 | 180.2739366 | 133.1106471 | 16.90691755 | -155.2452746 | 1.410477266 | -1.569251836 | 0.738101525 | 1.325632577 | -0.612472 |
| 22 | 37.07540625 | -261.1124001 | 369.418646 | -76.6639384 | 0.612025244 | -87.15608327 | -1.344400463 | 0.759169765 | -0.114177319 | -0.292800294 | 0.915760 |

*REAL VALUES*

```
Target: {'x': 37, 'y': 29, 'z': 418, 'roll': -34, 'pitch': -7, 'yaw': 40}
Predicted Joints (Degrees):
Q1: 40.51
Q2: -13.46
Q3: 11.03
Q4: -55.28
Q5: 83.92
```

*PREDICTED VALUES*




*Conversion from deg to rad*

## 11. Conclusion

The developed ANN successfully approximates the inverse kinematics of a five-degree-of-freedom robotic arm. By learning directly from workspace data, the model avoids complex analytical derivations and provides fast, accurate joint angle predictions suitable for real-time robotic applications such as painting robots and automated manipulators.